

UNIVERSITY OF HAIFA

Edge-Centric Intrusion Detection: A Critical Data Science Evaluation

Ilay Nadler

Supervisor: Dr. Uri Itai

July 2026

What Changed in This Version

This resubmission responds to the three points raised on the previous (graded) version. All three are addressed both in this report and in the accompanying, fully re-executed notebook.

- **1. Sequences are now built in capture order, before a temporal split.** Previously the $T = 10$ sliding windows were formed after a random stratified split, so adjacent rows in a window were not consecutive packets or one flow. Both experiments now preserve capture order and use a chronological train/validation/test split; each window is ten genuinely consecutive records and no window crosses a split boundary. This single fix materially changed the results and, as it turns out, reverses the previous submission's central conclusion.
- **2. The final test set is untouched.** Early stopping is monitored only on the validation segment; the test segment is a positional slice that is disjoint from train/validation by construction (asserted in code), is never passed to `model.fit()`, and is scored exactly once.
- **3. Restart-and-run-all with the corrected feature removal.** The notebook was executed top-to-bottom from a clean kernel, so every figure reflects the current constant/duplicate/Spearman-redundant feature removal. A capture-order index column (`ID_Column`) that had been silently retained as a CICIDS2017 feature is now dropped, because under a temporal split it leaks positional information about the label.

Abstract

This report critically evaluates the methodology and claims of “Edge-Detect: Edge-centric Network Intrusion Detection using Deep Neural Network” (Singh et al., 2021), which proposes a lightweight sliding-window recurrent network (RNN/LSTM/GRU) to detect DDoS attacks on resource-constrained edge devices, claiming up to 99% accuracy on UNSW-NB15. Reproduction from the authors’ own repository revealed that the artifact actually shipped for reproduction is not UNSW-NB15 but a specific, highly imbalanced capture from CICIDS2017. Two experiments are reported under a leakage-free, temporally-ordered methodology: a true reproduction on the official UNSW-NB15 partition (Experiment A) and an adaptation study on the released CICIDS2017 file (Experiment B). Under the corrected methodology the picture differs sharply from the previous submission: on UNSW-NB15 the sliding-window GRU reaches MCC 0.960 (98.0% accuracy) and clearly outperforms a stateless Random Forest (MCC 0.596), which collapses into a flood of false positives under the train-to-test temporal shift. On the CICIDS2017 file both models remain near-saturated ($\text{MCC} > 0.98$), an artifact of that file’s narrow single-afternoon capture. The corrected evaluation therefore partially supports the paper’s temporal architecture, while the undocumented dataset substitution, the attack-heavy prevalence of both benchmarks, and the absence of any real edge-hardware benchmark remain valid, serious criticisms.

Contents

0.1	Executive Summary	2
0.2	Summary of the Source	2
0.3	Critical Evaluation	3
0.4	Methodology: Temporal Windowing and Train/Validation/Test Separation . . .	3
0.5	Exploratory Data Analysis	4
0.5.1	Experiment A: UNSW-NB15	4
0.5.2	Experiment B: CICIDS2017	4
0.6	Feature Engineering Analysis	5
0.6.1	Experiment A: UNSW-NB15	5
0.6.2	Experiment B: CICIDS2017	5
0.7	Reproducibility Analysis	5
0.8	Experimental Results	5
0.8.1	Evaluation Metrics	5
0.8.2	Experiment A: UNSW-NB15 Results	6
0.8.3	Experiment B: CICIDS2017 Results	6
0.8.4	Cross-Experiment Comparison	7
0.9	Error Analysis	7
0.9.1	Experiment A: UNSW-NB15	7
0.9.2	Experiment B: CICIDS2017	7
0.10	Conclusions	8
0.10.1	Lessons Learned	8
0.11	Summing It Up	8
	Bibliography	10

0.1 Executive Summary

The paper proposes a lightweight sliding-window recurrent model for DDoS detection on edge devices and claims up to 99% accuracy on UNSW-NB15. Reproducing the work from the authors’ public repository revealed that the released reproducible artifact is not UNSW-NB15 at all, but the well-known CICIDS2017 `Friday-WorkingHours-Afternoon-DDos` capture. This undocumented mismatch is itself a central reproducibility finding, so this report evaluates both datasets separately.

Methodology. Every experiment uses an independent, chronological three-way split (70/15/15) on capture-ordered data. Sliding windows are built within each contiguous segment, so they are composed of consecutive records. The validation segment – not the test segment – drives early stopping; the test segment is scored once. The scaler is fit on training data only.

Key finding (revised). Fixing the windowing reverses the previous conclusion. On UNSW-NB15 the sliding-window GRU (MCC 0.960, accuracy 98.0%) now clearly beats the stateless Random Forest (MCC 0.596, accuracy 79.2%): coherent temporal windows carry a signal that the earlier shuffled windows destroyed, and the stateless RF fails to transfer across the temporal distribution shift, mislabelling $\sim 45\%$ of normal test traffic as attack. On CICIDS2017 both models stay near-perfect (GRU MCC 0.985, RF MCC 0.980), reflecting that file’s homogeneity rather than genuine model quality. Under this corrected evaluation the paper’s $\sim 99\%$ accuracy figure is far closer to replicating on its own stated dataset than the previous random-split analysis suggested.

Recommendation. The temporal architecture is better supported than previously concluded, but deploying this specific implementation is still not recommended without resolving and disclosing the dataset discrepancy, evaluating on realistic benign-heavy traffic, and benchmarking on actual constrained hardware.

0.2 Summary of the Source

The paper by Singh et al. addresses Network Intrusion Detection for edge-centric networks. An Intrusion Detection System (IDS) monitors activity inside a network and alerts on potential maliciousness; unlike a firewall’s static rules, it is dynamic. In large, distributed networks a lightweight, local IDS is attractive because data cannot move from every node to a central analyzer in reasonable time, and because behaviour that is benign in one segment may be threatening in another. The authors propose many small deep-learning models (RNN/LSTM/GRU) placed at the edges of the network to detect DDoS attacks quickly and with few resources, using a sliding window of $T = 10$ records to capture short-term temporal structure.

Stated dataset (per the paper): UNSW-NB15 (Moustafa & Slay, 2015), chosen for its modern traffic, clean training/testing separation, and labelled flow features (protocol, service, state, and byte/packet statistics).

Actual dataset released for reproduction: the authors’ repository ships `Friday-WorkingHours-Afternoon` from CICIDS2017 – 78 CICFlowMeter features plus a binary label, 225,745 rows – with no UNSW-NB15 file anywhere and no note explaining the substitution. This report therefore evaluates both: Experiment A uses the genuine UNSW-NB15 partition; Experiment B uses the CICIDS2017 file the authors’ code actually runs against.

0.3 Critical Evaluation

The core claim is that the proposed recurrent pipeline achieves highly reliable accuracy (up to 99%) while remaining small enough to run on constrained devices (e.g. a Raspberry Pi). Neither the paper nor this report actually deploys the models on such hardware; disk footprint and inference latency are used as proxies, measured on the training machine.

The dataset claim does not match the released artifact. A reader verifying the paper’s central claim using the authors’ own code and data would in practice be testing on CICIDS2017, not UNSW-NB15. This is why the two datasets are treated as separate exercises.

On the paper’s own stated dataset (Experiment A), the accuracy claim is now largely supported – by the GRU. Under a correct temporal evaluation the sliding-window GRU reaches 98.0% accuracy (MCC 0.960), approaching the paper’s $\sim 99\%$ figure, whereas the stateless Random Forest reaches only 79.2% (MCC 0.596). This contradicts the previous submission, which – using windows built on randomly shuffled rows – wrongly found the GRU weak and the RF strong. The earlier result was an artifact of an invalid windowing procedure: shuffling destroys the temporal adjacency the architecture is designed to exploit.

The stateless baseline fails under temporal shift. Trained on an attack-heavy earlier segment and evaluated on later traffic, the Random Forest flags $\sim 45\%$ of all normal test flows as attack. This is an important, realistic failure mode: a per-flow model calibrated on one period’s prevalence produces an alert flood when the traffic distribution drifts.

The released artifact (Experiment B) is too easy to be evidence of anything. The CICIDS2017 file is a single staged DDoS afternoon; both models remain near-saturated (MCC > 0.98) even after a temporal split that pushes its test-set attack prevalence down to 8.6%. Its near-ceiling scores reflect capture homogeneity, not generalization.

Overall. Under a methodologically sound evaluation the paper’s architectural choice – temporal sliding windows – is better justified than the previous analysis implied: it beats a strong stateless baseline and is more robust to temporal distribution shift. The valid criticisms that remain concern the undocumented dataset substitution, the attack-heavy (unrealistic) prevalence of both benchmarks, and the untested edge-hardware claim.

0.4 Methodology: Temporal Windowing and Train/Validation/Test Separation

Two methodological problems from the previous version are fixed together, because they interact.

Capture-order windows + temporal split. Neither released dataset carries a usable timestamp, but both store rows in capture/record order (UNSW-NB15’s dropped `id` column and CICIDS2017’s leading index column are monotonic record counters, verified in the notebook). Attack and normal traffic are strongly clustered along this order rather than interleaved. The data are therefore kept in capture order and split chronologically: the earliest 70% is training, the next 15% is validation, and the latest 15% is the test segment. The $T = 10$ windows are constructed within each contiguous segment, so every window is ten consecutive records and none straddles a split boundary. This is what the previous random-shuffle-then-window pipeline violated.

Leakage-free early stopping. The GRU is trained with `EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)` on the validation segment. The test segment is

a positional slice disjoint from train/validation (asserted in code), is never passed to `model.fit()`, and is used once for the metrics reported below. `StandardScaler` is fit on the training segment only and applied to validation/test. The Random Forest, needing no validation split, is trained on train+validation combined and evaluated on the identical held-out test segment as the GRU, keeping the comparison fair.

A direct consequence of temporal splitting is that class prevalence differs across segments – an honest reflection of training on past traffic and testing on future traffic:

Segment	UNSW-NB15 rows	UNSW attack %	CICIDS2017 rows	CICIDS attack %
Train	180,371	68.8%	157,997	64.0%
Validation	38,651	47.5%	33,857	70.7%
Test (held out)	38,651	57.4%	33,857	8.6%

0.5 Exploratory Data Analysis

0.5.1 Experiment A: UNSW-NB15

The combined official partition has 257,673 rows and 45 columns; no missing or infinite values were found, so no rows were dropped on that basis. No constant, duplicate-name, or exact duplicate-value columns exist among the native fields. Numeric flow features (byte/packet counts, inter-arrival times) are heavily right-skewed with long attack-driven tails; outliers are retained because they frequently carry the attack signal. Temporal (capture-order) analysis shows attack traffic arriving in long homogeneous runs rather than interleaved – the structure that makes capture-order windowing necessary. The combined partition is 63.9% attack overall, and the temporal test segment is 57.4% attack: far from the benign-heavy composition of real edge traffic. Spearman correlation ($|\rho| > 0.9$) flagged 22 redundant numeric features, leaving 21 numeric plus the categorical `proto`, `service` and `state` fields.

0.5.2 Experiment B: CICIDS2017

Loading the released files first required fixing a header-alignment bug: the shipped header lists one fewer name than the data has columns, and lists `Fwd Header Length` twice. After inserting the leading index-column name and de-duplicating, that index column (`ID_Column`) is dropped as a non-feature. Four rows carried conventional missing `Flow Bytes/s` values and several flow-rate columns contained `Infinity` from division-by-zero on zero-duration flows; these 34 rows were dropped (225,745 $\rightarrow$ 225,711). Numeric features are heavily right-skewed. The capture is 56.7% DDoS overall; under the temporal split the held-out segment is only 8.6% attack, because the file’s benign flows cluster at the end of the capture. Spearman correlation plus exact-redundancy checks flagged 48 features for removal, leaving 30.

Correlation choice. Spearman is used throughout because network-flow features are frequently monotonically but non-linearly related (e.g. packet counts vs. byte counts) and Spearman’s rank-based formulation is robust to the heavy-tailed outliers typical of burst traffic, without the small-sample bias that would favour Kendall’s τ .

0.6 Feature Engineering Analysis

0.6.1 Experiment A: UNSW-NB15

UNSW-NB15 contains genuine string categoricals: `proto`, `service` and `state`. One-Hot Encoding is therefore actually exercised here – 21 numeric features plus 3 categorical columns expand to 171 features, since `proto` and `service` take dozens of rare values. Standard (Z-score) scaling is applied after encoding so large-magnitude byte/packet features do not dominate gradient descent. The 22 Spearman-redundant numeric features are dropped before encoding, so redundant signal is not propagated into the one-hot expansion. A useful additional feature set would be rolling multi-flow statistics (e.g. distinct destination IPs contacted per source per window, or SYN-to-completed-handshake ratios), which give more direct evidence of a volumetric DDoS ramp-up than per-flow features alone.

0.6.2 Experiment B: CICIDS2017

This file is already fully numeric, so one-hot encoding finds nothing to encode (the logic is retained for parity). After dropping `ID_Column`, 48 constant/duplicate/Spearman-redundant features are removed, leaving 30. Standard scaling is applied for the same gradient-descent reason. The most important hidden preprocessing step is handling the division-by-zero `Infinity` values in flow-rate features; the affected rows are dropped rather than imputed, since a rate over a zero-duration flow has no clear interpretation.

0.7 Reproducibility Analysis

The official implementation is hosted at github.com/racsa-lab/Edge-Detect.

Dataset mismatch. The repository’s `dataset/` folder contains only the raw CICIDS2017 file, a header file, and a pre-processed `modified_dataset.csv` (used in Experiment B). There is no UNSW-NB15 file, and the README does not explain the substitution.

Execution and dependencies. For Experiment A the official UNSW-NB15 training/testing partition was obtained independently. For Experiment B the exact files released by the authors were used as-is. Both pipelines execute successfully in a Python/TensorFlow environment.

Hidden preprocessing. Beyond the dataset substitution, the released files hide a header-alignment bug and a monotonic index column that must not be used as a feature – exactly the kind of silent step that breaks naive reproduction.

Overall. The architecture is straightforward to reconstruct, but claim verification is compromised by the substitution: a reader following only the repository would silently reproduce results on a materially easier dataset without any indication that this had happened.

0.8 Experimental Results

0.8.1 Evaluation Metrics

Accuracy, macro Precision/Recall/F1 and ROC-AUC are reported, but MCC and the confusion matrix are the primary comparison metrics: MCC stays informative under class imbalance and near-saturation, and the confusion matrix grounds the error analysis. In an IDS a False Negative

lets an attack through (potential downtime), while excessive False Positives cause alert fatigue – so the balance of the confusion matrix, not headline accuracy, is what matters operationally.

0.8.2 Experiment A: UNSW-NB15 Results

Metric	GRU (Edge-Detect, $T=10$)	Random Forest
Accuracy	0.9803	0.7919
Precision (macro)	0.9791	0.8404
Recall (macro)	0.9807	0.7607
F1 (macro)	0.9799	0.7675
MCC	0.9598	0.5958
ROC-AUC	0.9973	0.9408
Disk footprint	0.26 MB	145.7 MB
Latency	0.155 ms/window	0.017 ms/packet

Confusion matrices (rows = actual, columns = predicted):

Model	TN (normal→normal)	FP (normal→attack)	FN (attack→normal)	TP (attack→attack)
GRU	16,210	270	493	21,669
Random Forest	9,042	7,438	605	21,566

The GRU outperforms the Random Forest on every predictive metric. The RF’s 7,438 false positives – 45% of all normal test traffic – are the signature of a stateless model failing to transfer across the temporal distribution shift. The GRU also keeps a decisive footprint advantage (0.26 MB vs. 145.7 MB), the latter inflated by the 171-feature one-hot input; the RF retains only a raw per-sample latency edge.

0.8.3 Experiment B: CICIDS2017 Results

Metric	GRU (Edge-Detect, $T=10$)	Random Forest
Accuracy	0.9976	0.9968
Precision (macro)	0.9930	0.9859
Recall (macro)	0.9915	0.9941
F1 (macro)	0.9923	0.9900
MCC	0.9846	0.9800
ROC-AUC	0.9986	0.9968
Disk footprint	0.11 MB	0.93 MB
Latency	0.148 ms/window	0.011 ms/packet

Model	TN	FP	FN	TP
GRU	30,902	36	46	2,864
Random Forest	30,857	81	27	2,892

Both models are near-saturated (GRU marginally ahead). Even after the temporal split lowers test-set attack prevalence to 8.6%, this file remains an easy task – a property of the single-afternoon capture, not the models.

0.8.4 Cross-Experiment Comparison

	Experiment A (UNSW-NB15)	Experiment B (CICIDS2017)
Test-set attack prevalence	57.4%	8.6%
GRU MCC	0.960	0.985
Random Forest MCC	0.596	0.980

The contrast is the finding. On the paper’s own dataset the temporal architecture is strongly justified and its accuracy claim nearly replicates; on the substituted file everything saturates regardless of model, so it tells us little. The gap between a 20.8% Random-Forest error rate on UNSW-NB15 and a 0.32% rate on CICIDS2017 measures dataset difficulty, not model quality.

0.9 Error Analysis

Because the Random Forest is the weaker model, its errors are the more instructive to dissect. Feature means are compared between misclassified (FP+FN) and correctly classified (TP+TN) samples, restricted to informative traffic features so sparse one-hot dummies do not dominate the ranking.

0.9.1 Experiment A: UNSW-NB15

The Random Forest misclassifies 8,043 of 38,651 test samples (20.81%), overwhelmingly as false positives: 7,438 normal flows flagged as attack versus only 605 missed attacks. The errors are structured, not random – destination TTL (`dttl`) is markedly higher among errors (mean ~ 182 for false positives) than among correctly classified normal traffic (~ 170) or correctly classified attacks (~ 106), and features such as `smean` and `response_body_len` also separate errors from correct predictions. Operationally this is the alert-fatigue failure mode: a per-flow model calibrated on a 68.8%-attack earlier segment defaults to “attack” once the traffic distribution shifts, drowning operators in false alarms. The GRU, by contrast, makes only 270 false positives and 493 false negatives on the same segment, because its ten-step temporal context contextualizes each flow.

0.9.2 Experiment B: CICIDS2017

Under the temporal split the Random Forest now makes 108 errors (81 FP, 27 FN) rather than the single error seen under the old random split – enough to analyze in aggregate. The strongest error-associated features are inter-arrival-time statistics (`Flow IAT Min`, `Fwd IAT Min`) and TCP flag counts (`FIN/ACK Flag Count`): the mistakes are boundary flows with atypical timing, not a systematic weakness. Given the near-total absence of errors, this describes a blind spot on this specific, homogeneous file rather than a generalizable pattern.

False-positive / false-negative trade-off. For an edge IDS, both benchmarks’ attack-heavy training prevalence biases a stateless model toward predicting “attack,” which would produce a cascade of false positives against realistic benign-heavy traffic – precisely what the Random Forest exhibits on the UNSW-NB15 temporal test segment. A temporal, context-aware model is both more accurate and better calibrated here, which is the strongest evidence in the paper’s favour.

0.10 Conclusions

Corrected central finding. Once sliding windows are built from consecutive records and the split is temporal and leakage-free, the paper’s sliding-window architecture is vindicated on its own stated dataset: the GRU reaches MCC 0.960 (98.0% accuracy) on UNSW-NB15 and outperforms a stateless Random Forest (MCC 0.596), which collapses into false positives under temporal shift. This reverses the previous submission’s conclusion, which was an artifact of windowing on shuffled rows.

The dataset substitution still matters. The paper claims UNSW-NB15 but its repository ships CICIDS2017, undocumented. On that substituted file both models are near-perfect ($\text{MCC} > 0.98$) regardless of methodology – a property of a narrow single-afternoon capture, not evidence for the paper’s claims.

Both benchmarks are attack-heavy. A model calibrated on either partition is not calibrated for the benign-heavy traffic a real edge network sees; the Random Forest’s 45% false-positive rate on the UNSW-NB15 temporal test segment is a concrete demonstration.

0.10.1 Lessons Learned

- Sequence models must be evaluated on genuinely ordered sequences: building windows after a random shuffle silently invalidates both the model and the comparison, and can flip the headline conclusion.
- A separate validation set is mandatory whenever early stopping or checkpointing is used; the test set must be scored exactly once.
- Verifying a paper’s claimed dataset against what its repository actually ships is a distinct, necessary reproducibility check – separate from checking that the code runs.
- Stateless per-flow models can look strong under a random split yet fail badly under a realistic temporal split; the split design is itself a first-class experimental decision.

0.11 Summing It Up

This project critically evaluated “Edge-Detect,” which proposes a lightweight sliding-window recurrent architecture for DDoS detection on edge devices, claiming up to 99% accuracy on UNSW-NB15. Reproduction surfaced an undocumented dataset mismatch (the repository ships a CICIDS2017 DDoS capture), so both datasets were evaluated under a corrected, leakage-free, temporally-ordered methodology.

On the paper’s actually-claimed dataset (UNSW-NB15), the sliding-window GRU reaches MCC 0.960 / 98.0% accuracy and clearly outperforms a stateless Random Forest (MCC 0.596), which floods false positives under temporal distribution shift – so the paper’s accuracy claim nearly replicates and its temporal architecture is justified. On the dataset actually released (CICIDS2017), both models are near-perfect ($\text{MCC} > 0.98$), reflecting that capture’s homogeneity rather than genuine model quality.

Do we recommend this project for similar problems? Partially. The temporal sliding-window approach is sound and, on properly ordered data, beats a strong stateless baseline while remaining small enough for edge use. But we would not deploy this specific implementation

without (i) resolving and disclosing the dataset discrepancy, (ii) evaluating on realistic benign-majority traffic rather than either attack-heavy benchmark, and (iii) benchmarking on the actual constrained hardware the paper targets – which neither the paper nor this report has done.

Scope limitation. Disk footprint and inference latency were measured on the training machine as proxies for edge feasibility, not on a physically constrained device. Real edge deployment (limited RAM/CPU, no GPU, power and thermal budgets) could shift the balance between the two models beyond what is reported here.

Bibliography

- [1] P. Singh, J. J. P. A. Pankaj, and R. Mitra, “Edge-Detect: Edge-Centric Network Intrusion Detection using Deep Neural Network,” *2021 IEEE 18th Annual CCNC*, pp. 1–6, 2021.
- [2] racsa-lab, “Edge-Detect” [GitHub repository], github.com/racsa-lab/Edge-Detect.
- [3] N. Moustafa and J. Slay, “UNSW-NB15: a comprehensive data set for network intrusion detection systems,” *2015 MilCIS*, pp. 1–6, 2015.
- [4] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” *ICISSP 2018*, pp. 108–116.
- [5] Wikipedia contributors, “Intrusion detection system,” *Wikipedia, The Free Encyclopedia*.